PANDAS CORE API - QUICK / BASICS / SERIES / MISSING

CONCEPT NAME -> syntax only | packed combined sheet

MISSING DATA

```
pd.isnull(x)
pd.notnull(x)
pd.isna(x)
pd.notna(x)
df.isnull().sum()
df.fillna(0)
df.ffill()
df.bfill()
df.interpolate()
df.dropna()
df.dropna(how='all')
df.dropna(thresh=N)
df['col'].sum(skipna=True)
```

BOOLEAN SUBSETTING

```
ages > ages.mean()
ages[ages > ages.mean()]
ages[mask]
scientists[scientists['Age'] >
   scientists['Age'].mean()]
scientists[(scientists['Age'] > 40) &
   (scientists['Occupation'] == 'Chemist')]
scientists.query('Age > 40 and Occupation ==
   "Chemist"')
```

READ CSV NA

```
pd.read_csv('survey_visited.csv')
pd.read_csv('survey_visited.csv',
   keep_default_na=False)
pd.read_csv('survey_visited.csv', na_values=[''],
   keep_default_na=False)
pd.read_csv('file.csv', na_values=[99])
keep_default_na=False
na_filter=False
```

COMBINING DATA

```
pd.concat([df1, df2])
pd.concat([df1, df2], axis=1)
pd.concat([df1, df2], ignore_index=True)
pd.concat([df1, df2], join='inner')
left.merge(right, on='key')
left.merge(right, left_on='a', right_on='b',
   how='outer')
```

DO NOT USE

```
df.shape()
df[0]
df[[0]]
df.loc[-1]
df['a', 'b']
.ix[]
```

DO NOT USE

```
nan == nan
x == np.nan
df.fillna(method='ffill')
df.fillna(method='bfill')
inplace=True
.ix[...]
```

IMPORTS

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

COUNTING

```
df['country'].nunique()
df.groupby('continent')['country'].nunique()
df['continent'].value_counts()
df.groupby('continent')['country'].count()
```

DATA LOADING

```
df = pd.read_csv('data.csv')
df = pd.read_csv('data.tsv', sep='\t')
df = pd.DataFrame({...})
```

MERGE NAN

```
visited.merge(survey, left_on='ident',
   right_on='taken')
```

INSPECTION

```
df.head()
df.head(n=10)
df.tail()
df.tail(n=1)
df.shape
df.shape[0]
df.shape[1]
df.columns
df.dtypes
df.info()
df.describe()
type(df)
```

SUBSETTING

```
df['col']
df[['col1', 'col2']]
df.iloc[:, n]
df.loc[label]
df.iloc[pos]
df.loc[row_sel, col_sel]
df[df['col'] > x]
df[(cond1) & (cond2)]
df.query('cond1 and cond2')
```

ROW SELECTION

```
df.loc[0]
df.loc[[0, 99, 999]]
df.iloc[0]
df.iloc[-1]
df.iloc[[0, 99, 999]]
df.iloc[0:3]
df[start:stop:step]
df[bool_series]
```

PLOTTING

```
df['col'].plot.hist()
df.plot.scatter(x='a', y='b')
sns.histplot(df['col'])
sns.scatterplot(x='a', y='b', data=df)
sns.boxplot(x='cat', y='num', data=df, hue='cat2')
sns.lmplot(x='a', y='b', data=df, col='facet_col')
fig, ax = plt.subplots()
```

DEPRECATED / REMOVED

```
.ix[...] -> .loc[...] / .iloc[...]
df.append(...) -> pd.concat([...])
sns.distplot(...) -> sns.histplot() / sns.kdeplot()
df.fillna(method='ffill') -> df.ffill()
df.fillna(method='bfill') -> df.bfill()
inplace=True -> df = df.method(...)
```

DTYPES

```
object
int64
float64
bool
datetime64[ns]
category
```

ROWS + COLUMNS

```
df.loc[42, 'country']
df.iloc[42, 0]
df.loc[[0,99], ['country','lifeExp']]
df.iloc[[0,99], [0,3]]
df.loc[row_sel, col_sel]
```

PANDAS OBJECTS

```
pd.Series(...)
pd.DataFrame(...)
type(df['country'])
type(df[['country']])
```

IMPORTS

```
import numpy as np
import pandas as pd
from numpy import nan
```

IMPORT

```
import pandas as pd
```

SERIES METHODS

```
first_row.keys()
s.mean()
s.median()
s.mode()
s.min()
s.max()
s.std()
s.var()
s.quantile(q)
s.describe()
s.sum()
s.count()
```

TIDY & RESHAPING

```
pd.melt(df, id_vars=['id'], var_name='var',
   value_name='val')
df.pivot_table(index='id', columns='var',
   values='val').reset_index()
df['col'].str.split('_', expand=True)
df[['a','b']].drop_duplicates()
df.merge(sub, on=['a','b'])
glob.glob('data/*.csv')
pd.concat([pd.read_csv(f) for f in files])
```

EXPORT / IMPORT

```
df.to_pickle('scientists_df.pickle')
pd.read_pickle('scientists_df.pickle')
df.to_csv('scientists.csv', index=False)
pd.read_csv('scientists.csv')
df.to_excel('scientists.xlsx', index=False)
pd.read_excel('scientists.xlsx')
df.to_parquet('scientists.parquet', index=False)
pd.read_parquet('scientists.parquet')
```

CREATING & MODIFYING

```
pd.Series([1,2,3], index=['a','b','c'])
pd.DataFrame({'col1': [...], 'col2': [...]})
df['new_col'] = ...
df.rename(columns={'old':'new'})
df.drop(columns=['col'])
df['col'].apply(func)
df.apply(lambda row: ..., axis=1)
```

COLUMN SELECTION

```
df['country']
df.country
df[['country','continent','year']]
df.iloc[:, 0]
df.iloc[:, [0, -1]]
df.iloc[:, 0:3]
```

SERIES UNIQUE / SORT

```
s.unique()
s.nunique()
s.value_counts()
s.sort_values()
s.sort_index()
s.drop_duplicates()
```

GROUPBY

```
df.groupby('year')
df.groupby('year')['lifeExp']
df.groupby('year')['lifeExp'].mean()
df.groupby(['year','continent'])[['lifeExp',
   'gdpPercap']].mean()
```

QUICK PLOT

```
import matplotlib.pyplot as plt
yearly_life_exp = df.groupby('year')['lifeExp'].mean()
yearly_life_exp.plot()
plt.show()
```

REINDEX NAN

```
life_exp = gapminder.groupby('year')['lifeExp'].mean()
y2000 = life_exp[life_exp.index > 2000]
y2000.reindex(range(2000, 2010))
```

DATETIME COLUMNS

```
scientists['born_dt'] =
   pd.to_datetime(scientists['Born'],
   format='%Y-%m-%d')
scientists['died_dt'] =
   pd.to_datetime(scientists['Died'],
   format='%Y-%m-%d')
scientists['age_days'] = scientists['died_dt'] -
   scientists['born_dt']
scientists['age_years'] =
   scientists['age_days'].dt.days // 365
```

NAN CHECKS

```
pd.isnull(nan)
pd.isnull(42)
pd.notnull(nan)
pd.notnull(42)
pd.isna(x)
pd.notna(x)
df.isna()
df.notna()
df.isnull()
df.notnull()
```

EXPORT / IMPORT

```
df.to_csv('f.csv', index=False)
pd.read_csv('f.csv')
df.to_excel('f.xlsx', index=False)
pd.read_excel('f.xlsx')
df.to_pickle('f.pickle')
pd.read_pickle('f.pickle')
df.to_parquet('f.parquet')
pd.read_parquet('f.parquet')
```

GROUPBY / AGGREGATION

```
df.groupby('col')['target'].mean()
df.groupby(['col1','col2'])[['t1','t2']].mean()
df.groupby('col').agg({'t1':'mean','t2':'sum'})
df.groupby('col').agg(avg=('t1','mean'), total=('t2',
   'sum'))
df['col'].nunique()
df['col'].value_counts()
```

CALCULATIONS

```
ebola.sum()
ebola.sum(skipna=True)
ebola.sum(skipna=False)
ebola.mean()
ebola.count()
ebola.min()
ebola.max()
```

SERIES ATTRIBUTES

```
first_row.index
first_row.values
first_row.dtype
first_row.shape
first_row.size
first_row.T
```

CREATE SERIES

```
pd.Series(['banana', 42])
pd.Series(['Wes McKinney','Creator of Pandas'],
   index=['Person','Who'])
pd.Series({'goat': 4, 'amoeba': np.nan})
pd.Series(5, index=['a','b','c'])
```

CREATE DATAFRAME

```
pd.DataFrame({'Name': [...], 'Age': [...]})
pd.DataFrame(data={...}, index=[...], columns=[...])
pd.DataFrame([['n1','n2','n3','n4']], columns=['A','B',
   'C','D'])
```

IMPORTS

```
import pandas as pd
import numpy as np
```

DO NOT USE

```
and / or / not on Series
inplace=True
```

SERIES TRANSFORM

```
s.isin([...])
s.apply(func)
s.map(func_or_dict)
s.astype(dtype)
s.replace(a, b)
s.sample(n)
s.to_frame()
s.corr(other)
s.cov(other)
s.equals(other)
```

LOAD & INSPECT

```
df = pd.read_csv('data.csv')
df = pd.read_csv('data.tsv', sep='\t')
df.head()
df.tail()
df.shape
df.columns
df.dtypes
df.info()
df.describe()
```

MODIFY DATAFRAME

```
df['new_col'] = ...
sorted_ages = ages.sort_values()
scientists['Age'] =
scientists['Age'].sample(frac=1).reset_index(drop=True)
df.rename(columns={'old_name':'new_name'})
df.drop(columns=['col_to_remove'])
df[['col_b','col_a']]
df['col'].apply(lambda x: x * 2)
df.apply(lambda row: row['a'] + row['b'], axis=1)
```

COUNT MISSING

```
ebola.shape
ebola.count()
ebola.shape[0] - ebola.count()
np.count_nonzero(ebola.isnull())
np.count_nonzero(ebola['Cases_Guinea'].isnull())
ebola['Cases_Guinea'].value_counts(dropna=False).head()
df.isnull().sum()
df.isna().sum()
```

AGGREGATION

```
df.groupby('continent').agg({'lifeExp':'mean',
   'pop':'sum'})
df.groupby('continent')['lifeExp'].agg(['mean','min',
   'max','count'])
df.groupby('continent').agg(avg_life_exp=('lifeExp',
   'mean'), total_pop=('pop','sum'))
```

FILL MISSING

```
ebola.fillna(0)
ebola = ebola.fillna(0)
ebola.ffill()
ebola.bfill()
ebola.interpolate()
ebola.interpolate(method=...)
```

DROP MISSING

```
ebola.dropna()
ebola.dropna(how='any')
ebola.dropna(how='all')
ebola.dropna(thresh=10)
ebola.dropna().shape
```

VECTORIZED OPERATIONS

```
ages + ages
ages * 2
pd.Series([1,2,3]) + pd.Series([10,20])
ages.values + np.array([...])
```

&

```
|
~
```

CREATE NAN

```
pd.Series({'goat': 4, 'amoeba': np.nan})
scientists['missing'] = np.nan
```

PANDAS VISUAL + COMBINE + RESHAPE API

CONCEPT NAME -> syntax only | packed combined sheet

SEABORN BIVARIATE

```
sns.scatterplot(x='total_bill', y='tip',
   data=tips)
sns.regplot(x='total_bill', y='tip', data=tips)
sns.regplot(x='total_bill', y='tip', data=tips,
   fit_reg=False)
sns.jointplot(x='total_bill', y='tip',
   data=tips)
sns.jointplot(x='total_bill', y='tip',
   data=tips,
   kind='hex')
sns.jointplot(x='total_bill', y='tip',
   data=tips,
   kind='kde')
sns.barplot(x='time', y='total_bill',
   data=tips)
sns.barplot(x='time', y='total_bill',
   data=tips,
   estimator='std')
sns.boxplot(x='time', y='total_bill',
   data=tips)
sns.violinplot(x='time', y='total_bill',
   data=tips)
```

RESHAPING PAIRS

```
wide -> pd.melt(...)
long -> df.pivot_table(...)
split -> s.str.split(...)
attach -> pd.concat(..., axis=1)
dedupe -> df.drop_duplicates()
link -> df.merge(...)
```

CONCAT COLUMNS

```
pd.concat([df1, df2, df3], axis=1)
col_concat['A']
col_concat['new_col'] = ['n1','n2','n3','n4']
pd.concat([df1, df2], axis=1,
   ignore_index=True)
```

SUFFIXES

```
left.merge(right, on='key', suffixes=('_left',
   '_right'))
left.merge(right, left_on='a', right_on='b',
   suffixes=('_x','_y'))
```

MERGE BASIC

```
left.merge(right, on='shared_col')
left.merge(right, left_on='name',
   right_on='site')
```

IMPORTS

```
import pandas as pd
import glob
```

FACETS

```
sns.lmplot(x='x', y='y', data=anscombe,
   fit_reg=False,
   col='dataset', col_wrap=2)
sns.lmplot(x='total_bill', y='tip', data=tips,
   fit_reg=False, hue='sex', col='day')
facet = sns.FacetGrid(tips, col='time')
facet.map(sns.histplot, 'total_bill')
facet = sns.FacetGrid(tips, col='day',
   hue='sex')
facet.map(plt.scatter, 'total_bill', 'tip')
facet.add_legend()
facet = sns.FacetGrid(tips, col='time',
   row='smoker',
   hue='sex')
sns.catplot(x='day', y='total_bill', hue='sex',
   data=tips, row='smoker', col='time',
   kind='violin')
```

MULTIPLE FILES

```
glob.glob('data/*.csv')
files = glob.glob('data/*.csv')
pd.read_csv(f)
dfs = [pd.read_csv(f) for f in files]
pd.concat(dfs)
pd.concat([pd.read_csv(f) for f in files],
   ignore_index=True)
```

STYLES

```
sns.set_style('whitegrid')
sns.set_style('darkgrid')
sns.set_style('dark')
sns.set_style('white')
sns.set_style('ticks')
```

CONCAT LABELS

```
pd.concat([df1, df2, df3], join='outer')
pd.concat([df1, df2, df3], join='inner')
pd.concat([df1, df3], join='inner')
pd.concat([df1, df2, df3], axis=1)
pd.concat([df1, df3], axis=1, join='inner')
```

MELT BASIC

```
pd.melt(pew, id_vars='religion')
pd.melt(pew, id_vars='religion',
   var_name='income',
   value_name='count')
```

ONE-TO-ONE

```
visited_subset = visited.loc[[0, 2, 6], :]
site.merge(visited_subset, left_on='name',
   right_on='site')
```

SPLIT STRINGS

```
variable_split =
   ebola_long['variable'].str.split('_')
ebola_long['status'] =
   variable_split.str.get(0)
ebola_long['country'] =
   variable_split.str.get(1)
variable_split =
   ebola_long['variable'].str.split('_',
   expand=True)
variable_split.columns = ['status','country']
pd.concat([ebola_long, variable_split], axis=1
ebola_long['status'], ebola_long['country'] =
   zip(*ebola_long['variable'].str.split('_'))
```

PIVOT TABLE

```
weather_melt.pivot_table(index=['id','year','m
   'day'], columns='element', values='temp')
weather_melt.pivot_table(..., aggfunc='mean')
weather_tidy =
   weather_melt.pivot_table(...).reset_index()
df.pivot_table(index='id', columns='var',
   values='val').reset_index()
```

SEABORN UNIVARIATE

```
sns.histplot(tips['total_bill'])
sns.histplot(tips['total_bill'], kde=True)
sns.kdeplot(tips['total_bill'])
sns.rugplot(tips['total_bill'])
sns.countplot(x='day', data=tips)
```

MATPLOTLIB PLOTS

```
ax.hist(tips['total_bill'], bins=10)
ax.scatter(tips['total_bill'], tips['tip'])
ax.boxplot([...], labels=[...])
ax.scatter(x=..., y=..., s=..., c=...,
   alpha=0.5)
```

INDEX / LABEL CHECKS

```
row_concat.iloc[3]
row_concat.loc[0]
df1.columns = ['A','B','C','D']
df1.index = [0,1,2,3]
```

MELT WEATHER

```
weather_melt = pd.melt(weather,
   id_vars=['id','year',
   'month','element'], var_name='day',
   value_name='temp')
```

DO NOT USE

```
sns.distplot(...)
```

NORMALIZE

```
billboard_songs =
   billboard_long[['year','artist',
   'track','time']].drop_duplicates()
billboard_songs['id'] =
   range(len(billboard_songs))
billboard_ratings =
   billboard_long.merge(billboard_songs,
   on=['year',
   'artist','track','time'])
billboard_ratings = billboard_ratings[['id',
   'date.entered','week','rating']]
```

AX METHODS

```
ax.plot(x, y)
ax.plot(x, y, 'o')
ax.set_title('Title')
ax.set_xlabel('x')
ax.set_ylabel('y')
fig.suptitle('Title')
fig.tight_layout()
plt.show()
```

MANY-TO-MANY

```
visited.merge(survey, left_on='ident',
   right_on='taken')
site.merge(visited, left_on='name',
   right_on='site').merge(survey,
   left_on='ident',
   right_on='taken')
```

CONCAT ROWS

```
pd.concat([df1, df2, df3])
pd.concat([df1, df2, df3], ignore_index=True)
pd.concat([df1, new_row_df])
pd.concat([df1, pd.DataFrame([data_dict])],
   ignore_index=True)
```

MANY-TO-ONE

```
site.merge(visited, left_on='name',
   right_on='site')
survey.merge(person, left_on='person',
   right_on='ident')
```

IMPORTS

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

DO NOT USE

```
df.append(df2)
df1.append(data_dict, ignore_index=True)
```

PANDAS PLOT

```
df['col'].plot()
df['col'].plot.hist()
df.plot.scatter(x='a', y='b')
df.plot.box()
df.plot.bar()
df.plot.line()
df.plot.area()
df.plot.density()
df.plot.hexbin(x='a', y='b')
```

MULTIVARIATE

```
sns.violinplot(x='time', y='total_bill',
   hue='sex',
   data=tips, split=True)
sns.lmplot(x='total_bill', y='tip', data=tips,
   hue='sex', fit_reg=False)
sns.lmplot(..., markers=['o','x'],
   scatter_kws={'s':
   tips['size'] * 10})
```

PAIRWISE

```
sns.pairplot(tips)
sns.pairplot(tips, hue='sex')
grid = sns.PairGrid(tips)
grid.map_upper(sns.scatterplot)
grid.map_lower(sns.kdeplot)
grid.map_diag(sns.histplot)
```

FIGURE / AXES

```
fig = plt.figure()
ax = fig.add_subplot(1, 1, 1)
fig, ax = plt.subplots()
fig, axes = plt.subplots(2, 2, figsize=(8, 8))
axes[0, 0].plot(...)
```

MERGE HOW

```
left.merge(right, on='key', how='inner')
left.merge(right, on='key', how='left')
left.merge(right, on='key', how='right')
left.merge(right, on='key', how='outer')
```

MELT MULTI ID

```
pd.melt(billboard,
   id_vars=['year','artist','track',
   'time','date.entered'], var_name='week',
   value_name='rating')
```

LOAD DATA

```
anscombe = sns.load_dataset('anscombe')
tips = sns.load_dataset('tips')
```

IMPORT

```
import pandas as pd
```